EXPERIMENT NO: 1

Name of the student: Adithya Menon **Roll No: 15**

Division: B **Batch:** A

Aim of the Experiment: Design and Implementation of a product cipher using Substitution and Transposition.

Learning Objectives: To understand the encryption and decryption fundamentals.
To understand the concepts of the product cipher.

Lab Outcomes: Understand the principles and practices of cryptographic techniques.

Tools & Software Required: Python

Theory:

(Explain following terms with one example)

Substitution cipher:A substitution cipher is a method of encryption where each letter (or symbol) in the plaintext is replaced with another letter (or symbol) according to a fixed system. The order of letters stays the same, only their identities change.

Example:

Plaintext: HELLO

Using a simple substitution where A→X, B→Y, C→Z, ..., H→M, E→J, L→O, O→T

Ciphertext: MJOOT

Transposition cipher:

A transposition cipher rearranges (shuffles) the letters of the plaintext according to a specific system, but does not replace letters with different letters. The same letters appear

in the ciphertext — just in a different order.

Example:

Plaintext: SECRET

Write in 2 rows (simple rail fence transposition): S C E E R T

Read column by column → SECRE T → ciphertext: SCREET

Caesar Cipher:

The Caesar cipher is a special case of a substitution cipher in which each letter in the plaintext is shifted by a fixed number of positions down or up the alphabet. It uses a single, constant shift value (the key).

Example:

Plaintext: HELLO

Key = +3

H→K, E→H, L→O, L→O, O→R

Ciphertext: KHOOR

Columnar Cipher:

A columnar transposition cipher is a transposition method that writes the plaintext into a grid row by row using a certain number of columns (the key), and then reads it out column by column (usually from left to right).

Example:

Plaintext: ATTACKATDAWN (remove spaces)

Key = 4 columns

Grid (written row-wise): A T T A C K A T D A W N

Read column-wise: ACDA TKTA TWAN → Ciphertext: ACDATKTATWAN

Algorithm/Procedure:

Part A: Caesar Cipher (Substitution)

1. Accept plaintext from the user.
2. Accept Caesar key from the user.
3. Take k = 0.
4. Extract the kth character from the plaintext string.
5. Add the key to the extracted character and obtain a new value.

   If the new value > 26,

   New value = New value % 26.
6. Add the obtained character as the kth character of the intermediate ciphertext.
7. Increment k.
8. If (k < length(plaintext)) go to step 4.
9. Store the result as intermediate ciphertext.

Part B: Columnar Transposition Cipher

1. Accept columnar key (number of columns) from the user.
2. Write the intermediate ciphertext row-wise into a matrix using the columnar key.
3. Read the characters column-wise from the matrix.
4. Store the obtained characters as final ciphertext.
5. Display plaintext and final ciphertext (output).

Source Code:

Caesar Cipher :

```
def caesar_encrypt(plaintext, key):
intermediate_ciphertext = ""
k = 0
while k < len(plaintext):
char = plaintext[k]
if char.isalpha():
base = ord('A') if char.isupper() else ord('a')
shifted = (ord(char) - base + key) % 26
new_char = chr(base + shifted)
intermediate_ciphertext += new_char
else:
intermediate_ciphertext += char
k += 1
```

```
return intermediate_ciphertext



if __name__ == "__main__":
print("=== Caesar Cipher Encryption ===\n")
text = input("Enter plaintext: ")
try:
shift = int(input("Enter Caesar key (shift value): "))
except ValueError:
print("Invalid key! Using default shift = 3")
shift = 3
result = caesar_encrypt(text, shift)
print("\nResults:")
print("Plaintext :", text)
print("Key :", shift)
print("Ciphertext :", result)

Output :
```

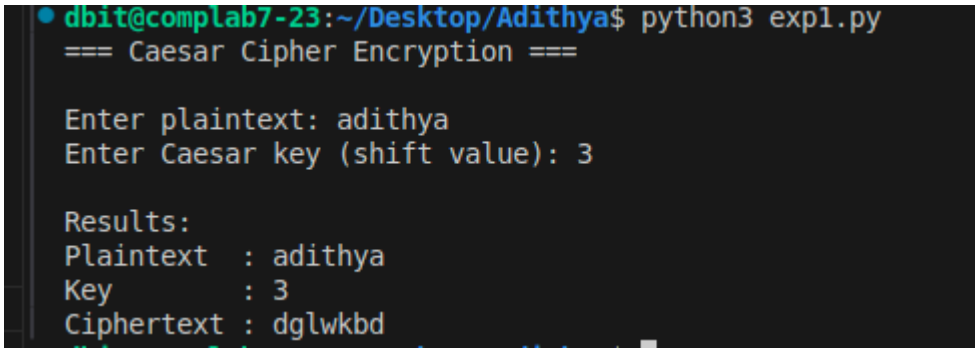
Columnar Cipher :

```
def columnar_encrypt(plaintext, cols):

plaintext = plaintext.replace(" ", "").upper()
length = len(plaintext)
rows = (length + cols - 1) // cols
matrix = [['' for _ in range(cols)] for _ in range(rows)]
idx = 0
for i in range(rows):
for j in range(cols):
if idx < length:
matrix[i][j] = plaintext[idx]
idx += 1
ciphertext = ''
for j in range(cols):
for i in range(rows):
if matrix[i][j]:
ciphertext += matrix[i][j]
return ciphertext

plaintext = input("Enter plaintext: ")
```

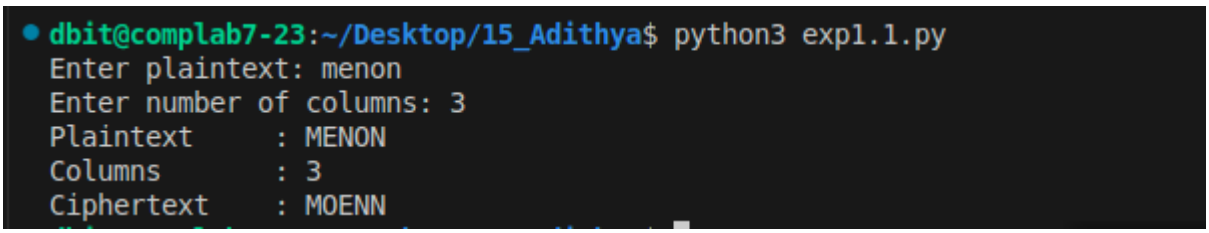
```
try:
columns = int(input("Enter number of columns: "))
if columns < 1:
columns = 1
except:
columns = 4

result = columnar_encrypt(plaintext, columns)

print("Plaintext :", plaintext.upper())
print("Columns :", columns)
print("Ciphertext :", result)
```

Output Screenshots:

Caesar Cipher :



Columnar Cipher :



Conclusion :

In conclusion, the Caesar cipher and the columnar transposition cipher represent two fundamental yet distinctly different classical encryption techniques that illustrate the core building blocks of cryptography: substitution and transposition.

The Caesar cipher, a simple monoalphabetic substitution method, shifts every letter in the plaintext by a fixed number of positions in the alphabet. While extremely easy to understand and implement, it offers almost no real security in practice. With only 26 possible keys (in the English alphabet), it can be broken almost instantly through brute-force enumeration or frequency analysis, as the letter distribution patterns remain clearly visible in the ciphertext.

In contrast, the columnar transposition cipher rearranges the letters of the plaintext according to a grid structure defined by a chosen number of columns (or sometimes a keyword), writing the message row-by-row and reading it column-by-column. This approach preserves the original letter frequencies but destroys the positional relationships and word patterns of the plaintext, making it significantly more resistant to simple frequency analysis than substitution ciphers like Caesar. A single columnar transposition is still vulnerable to attacks that involve guessing the number of columns, looking for anagrams, and reconstructing probable word structures — but it generally requires more effort and cryptanalytic skill to break than a Caesar cipher.